Inside the Serial Monitor window, we have a large central white space that displays whatever information it receives from the serial port. We also have a text entry area at the top that would allow us to send data out to a serial device. Along the bottom of the window is a check box to enable the Autoscroll feature, an option for the end-of-line character when sending data from the Serial Monitor, and a setting for the speed of the communications between the two serial devices. While we will discuss serial communications in much greater depth in Chapter 10, now that we have the code uploaded to the interface board, and the Serial Monitor is open, let's briefly look at how our code works.

How It Works

To get the Serial library working with the Arduino interface board, and for it to send data through the hardware serial port, we need to first establish the communications speed between the two devices using the `Serial.begin()` function. This is generally done inside the `setup()` function, as was shown in our example code with the following line:

```
Serial.begin(9600);
```

Here, we tell the Arduino board at what speed to establish communications; in this case, our communication speed has been set to 9600 bits per second (bps or baud), a fairly common communication speed for these purposes. Basically, this helps the two digital devices agree on how fast they are going to talk to one another and this matching speed setting should also be selected in the Serial Monitor.

The text in the main window in Figure 6-8 that displays "analog value = 506" is created by the following two lines of code in the `loop()` function:

```
Serial.print("analog value = " );
Serial.println(analogValue, DEC);
```

The first line uses the `Serial.print()` function to tell the Arduino to send down the serial connection the characters `"analog value = "` with the double quotations signifying that the string of text should be displayed exactly as written. The second line uses the function `Serial.println()`, which is very similar to the first, but inserts a line return after it has sent its information so that the next time we send up a value it will start on the next line. This line will display the current value of the variable `analogValue` in decimal format (`DEC`).

That's about all there is to this simple little sketch. Now, with an analog sensor wired up to pin A0, the source code uploaded to the Arduino, and the Serial Monitor up and running, we should see a long list of analog values being updated every eighth of a second. This is pretty useful for what we are going to do with these values next.

Mapping Values

Once we see what kinds of values our analog sensor can give us, we can then decide what to do with those numbers. Sometimes we might want to drive a PWM pin using the analog input and so will need to change from a 10-bit value to an 8-bit one. Maybe instead of receiving values that we think we should get, like 0 to 1023, we get something more like 12 for the lowest number and 861 for the highest. It's even possible that although we have all these numbers, all we want to do are a few simple things depending on certain thresholds being reached. To make these unwieldy numbers more manageable, we can use several Arduino functions to map the values we get to the values that we want. Let's keep the photocell wired up and look at these functions in more detail.